Does It Actually Do That?

Causal methods, and the standards of evidence

Where we stopped

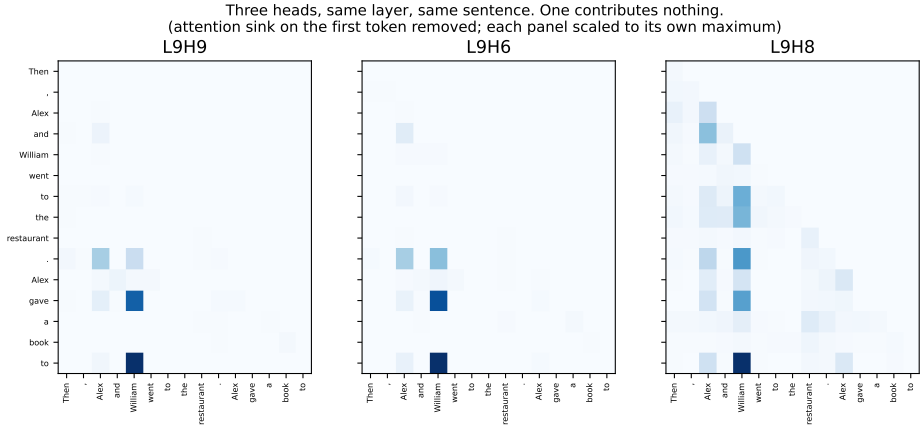
Last lecture gave us four ways to look inside a model:

the logit lens	the answer appears at layer 9
linear probes	duplication is readable from layer 3
attention patterns	heads look at the right names
max-activating examples	a component gets a name

Every one of them is a way of **watching**. Not one of them is a way of **checking**.

Today we stop watching and start **breaking things on purpose**.

Three heads. One of them is useless.



All three sit in **layer 9**. All three attend to " William", the correct answer. Exactly one of them contributes **nothing at all** to the model's output.

Pick one

Which of the three does nothing?

L9H9

sharp, focused on the answer

L9H6

sharp, focused on the answer

L9H8

looks at the answer from *everywhere*

Pick one

Which of the three does nothing?

L9H9
sharp, focused on the answer

L9H6
sharp, focused on the answer

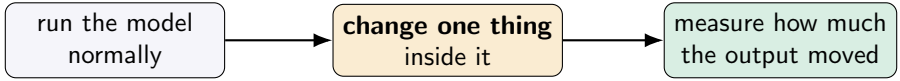
L9H8
looks at the answer from *everywhere*

Most people pick one of the two sharp ones, on the grounds that the third looks "messy". Some pick the third for the same reason. **Both arguments are made from the picture, and the picture does not contain the answer.**

We will settle it about halfway through the lecture - with a number, not a picture.

What we are about to do differently

Every method today has the same shape:



This is a **controlled experiment**, run on a network instead of on people. The model is a perfect subject: it has no memory between runs, no fatigue, and we can intervene anywhere we like.

Everything else today is a variation on *which* one thing you change.

Outline

Ablation

Delete a component. See what breaks.

The oldest idea in neuroscience

Remove a part, observe what stops working, conclude the part was responsible.

In a network the removal is easy and exact: take a head's output and **replace it** before it reaches the residual stream.

> You already know this move. In chapter 5, **permutation importance** shuffled an input column and watched the score fall. This is the same logic, one layer deeper: the thing being knocked out is now *inside* the model.

The only question is what to replace it *with*. It sounds like a detail. It is not.

Predict first

We take the three heads that contribute most to the answer - the ones we will name later - and delete them.

The logit difference is $+3.60$. Where does it go?

down to roughly zero down a little somewhere else

Hold your answer. We will get three different ones on the next few slides - and that is the point.

Three ways to delete a head

	What you replace it with	What you are asking
zero-ablation	the number 0	what if this head output nothing?
mean-ablation	its average output over many sentences	what if this head ignored *this* sentence?
resample-ablation	its output on a *different* sentence	what if this head were reading someone else's input?

These sound like three flavours of the same thing.

They are three **different questions**, and on our task they give three different answers - including different *signs*.

Why zero is not neutral

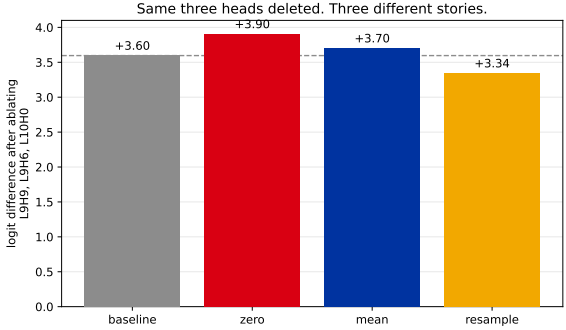
Zero feels like the obvious choice: switch the head off, set its contribution to nothing.

But a head's output is **never** zero in normal operation. Setting it to zero does not give the model "no information from this head" - it gives the model a vector it has **never seen in training**.

The model then behaves strangely, and you record that strangeness as the head's importance. You have measured your own corruption.

An analogy: to find out whether a violinist matters to an orchestra, you can ask them to stop playing - or you can replace them with someone playing *static*. Only one of those measures the violinist.

Same three heads, three answers



baseline	+3.60
zero	+3.91
mean	+3.70
resample	+3.34

Deleting L9H9, L9H6 and L10H0 - the three largest positive contributors. Two methods say the model got **better**. One says **worse**.

If a paper says "we ablated the head and performance dropped", your first question is **which kind of ablation**. The word alone does not identify the experiment.

And something else should be bothering you

We deleted the three most important heads in the model,
and by two of the three measures it **got better**.

That is not what "most important" is supposed to mean.

Park it. It is the last section of this lecture, and it turns out to be the most important result in it.

Activation patching

Instead of destroying information, move it

The problem with deleting

Ablation answers "what happens if this component is broken?"

That is rarely the question we care about. We want to know **which piece of information** is being carried, and **where**.

So instead of destroying a component's output, **swap in the output it would have had on a different input**. Nothing is broken. One specific piece of information is changed, and everything else is left alone.

This is the single most used technique in the field. It has two names - **activation patching** and **causal tracing** - which mean the same thing.

Two sentences, one word apart

Clean - the model gets it right:

> *"Then, Alex and William went to the restaurant.* **Alex** *gave a book to . . . "* $\rightarrow$ *" William"* $(+3.52)$

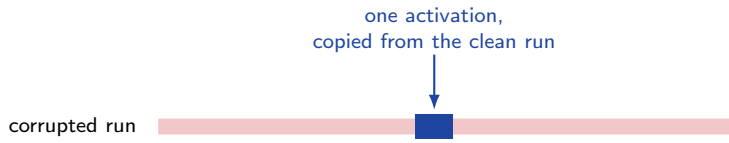
Corrupted - the names are swapped, so the correct answer flips:

> *"Then,* **William** *and* **Alex** *went to the restaurant.* **William** *gave a book to . . . "* $\rightarrow$ *" Alex"* (-4.14)

Measured against the *clean* answer, the model goes from $+3.52$ to -4.14. That gap of **7.66** is our working space: every patch we make recovers some fraction of it.

The move, step by step

1. Run the **clean** sentence. Save every activation.
2. Run the **corrupted** sentence. It gets the answer wrong.
3. Run the corrupted sentence **again** - but at one chosen layer and one chosen position, overwrite the activation with the value saved in step 1.
4. Measure the logit difference.



If that single value is enough to bring the correct answer back, then **that layer, at that position, is where the information lives**.

One patch, by hand

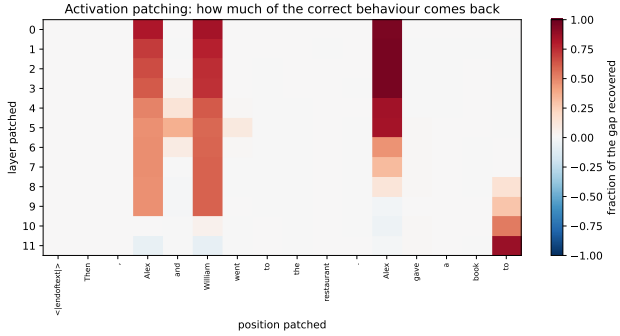
Patch the residual stream at **layer 1**, at the position of the second name:

corrupted run, no patch	-4.14
corrupted run, one activation patched	$+3.14$
recovered	7.28
the full gap	7.66
fraction recovered	**0.95**

95% of the model's correct behaviour, restored by copying one vector.
Everything else in the run is still corrupted - and it barely matters.

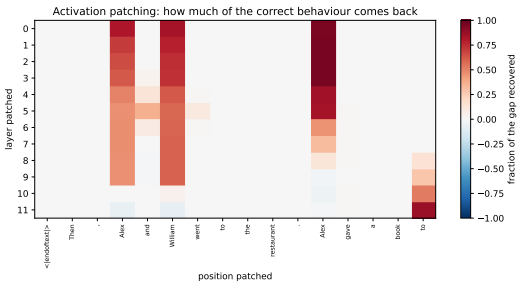
Now do that for every layer and every position, and put the answers in a grid.

The whole grid



Each cell: patch the residual stream at that layer and position, measure the fraction of the gap recovered. Dark red = the information is here.

Reading it



Early layers, at the two name positions. Patching recovers almost everything - fix the names early and the whole circuit runs correctly.

The signal then moves. By layer 8 the name positions stop mattering.

Late layers, final position. The last column lights up: the answer has been assembled and now lives where the prediction is made.

Read left to right and the picture shows **information travelling**: from the names, into the middle of the network, out at the final token.

What the grid cannot tell us

The grid is **layers and positions**. It says where the information is, and it says nothing about *which heads* carried it there.

We know the work happens around the name positions in early layers and arrives at the final token late. We still cannot name a single component.

Layer 9 was where lecture 1's logit lens said the answer appears. The next section finds out **who**, by a method that never mentions layers at all.

Two directions, two questions

We patched **clean into corrupted**. You can also go the other way.

	Denoising	**Noising**
Direction	clean value into a corrupted run	corrupted value into a clean run
Asks	is this **enough** to fix it?	is this **necessary**?
Finds	sufficient components	necessary components

These are **different questions** and they can give different answers. A redundant component is necessary-free but sufficient; a bottleneck is necessary but not sufficient alone. Papers routinely report one and describe it in the language of the other.

The clearest treatment is Heimersheim & Nanda, *How to Use and Interpret Activation Patching*.

Direct logit attribution

The one measurement that needs no intervention

We already built this

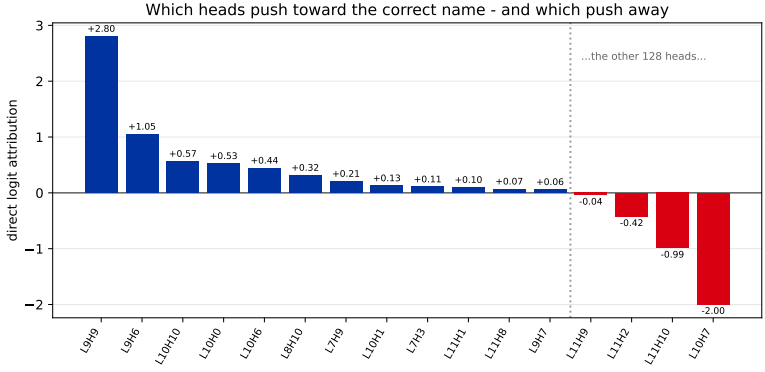
Last lecture we split the logit difference into one number per component, and checked that the numbers add up exactly:

$$\text{logit difference} = c_{\text{embed}} + c_{\text{head 0.0}} + \cdots + c_{\text{head 11.11}} + \cdots$$

Each c is that component's **direct logit attribution**: how hard it pushed toward the correct name.

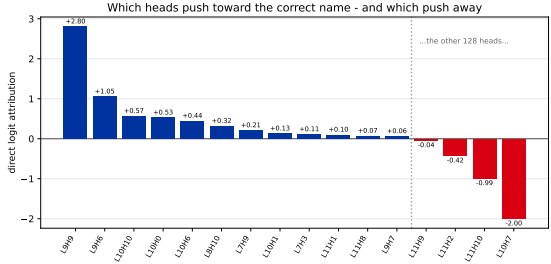
> No ablation, no patching, no second forward pass. One run, and every component's contribution falls out of the arithmetic - because the residual stream is a sum.

Every head, ranked



The 12 largest contributors and the 4 most negative, out of 144 heads. Blue pushes toward the correct name, red pushes away.

Two things in that chart are strange



One. The work is done by a **handful** of heads - L9H9 alone accounts for $+2.80$ of a $+3.60$ answer. Most of the 144 do essentially nothing.

Two. Some heads are **large and negative**. L10H7 contributes -2.00: it is actively pushing toward the *wrong* name.

A head whose job appears to be making the model worse is not a bug and not noise. It is the key to the last section.

Now we can settle the cold open

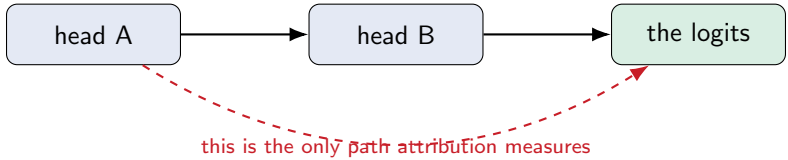
head	attention to the answer	direct logit attribution	effect of deleting it
L9H9	0.749	+2.805	+0.471
L9H6	0.664	+1.053	−0.522
L9H8	**0.318**	**−0.001**	**−0.034**

L9H8 attends to the correct answer from nearly every position in the sentence - the most visually striking "this head is about William" pattern of the three - and its contribution is $-\mathbf{0.001}$. Not small. **Nothing.**

Attention says where a head **looked**. It says nothing about what it **wrote**, and nothing about whether anything downstream **read** it.

What direct attribution cannot see

Direct logit attribution measures the **direct** path from a component to the output. That word is load-bearing.



If head A's entire job is to **set up** head B, its direct attribution is **zero** - and it is essential. Every head in the first half of our chart could be invisible to this method.

That gap is exactly what the next section exists to close.

Searching at scale

144 heads is small. What happens at 70 billion parameters?

Path patching

Ordinary patching replaces a **component's** output, which changes everything downstream at once.

Path patching replaces a component's contribution **along one route only** - head A's effect *on head B*, leaving head A's other effects untouched.

> This is what lets you say "head A matters *because of* head B" rather than just "head A matters". It is how the indirect, set-up-work heads get found.

The cost: the number of paths grows much faster than the number of components.

The arithmetic that forces the next idea

Our patching grid was 12 layers $\times$ 16 positions $=$ **192 forward passes**. On a laptop, about three minutes.

	components	a full sweep
GPT-2 small	144 heads	minutes
a 70B model	thousands of heads, long prompts	wildly impractical

Patching is exact and it does not scale. So the field did the obvious thing: approximate it.

Attribution patching

Take a first-order Taylor approximation of what patching would have done. The gradient tells you, to first order, how much the output would move if an activation changed.

One backward pass estimates every patch at once. Thousands of forward passes collapse into a single gradient computation - the same backpropagation from chapter 5, used to ask a different question.

It is an **approximation**, and it is worst exactly where the effect is large - which is where you were looking. Screen with attribution patching, then confirm the survivors with real patching.

Nanda (2023); evaluated systematically by Kramár et al. (Google DeepMind).

Letting the search run itself

Automated Circuit Discovery (ACDC) takes the loop we have been running by hand - patch a component, see whether the behaviour survives, drop it if it does - and automates it into a search over the whole network.

Give it a task, a metric and a model; get back a subgraph.

It recovers known circuits reasonably well and it is not magic: the results depend heavily on the metric you choose and the threshold you set. It narrows the search. It does not do the understanding.

Conmy et al. (2023).

The circuit, end to end

Everything so far, assembled into one answer

The task, one more time

> *"Then, Alex and William went to the restaurant. Alex gave a book to ..."*

Called **Indirect Object Identification (IOI)**: two names appear, one repeats, and the answer is the one that did not.

In 2022 a team at Redwood Research and Berkeley reverse-engineered how GPT-2 small does this - end to end, with the methods in this lecture.

Their answer: **26 attention heads**, in **7 classes**. Out of 144 heads total.

Wang, Variengien, Conmy, Shlegeris & Steinhardt (2022), *Interpretability in the Wild*.

The algorithm, in plain words

Find the name that appears **twice**.

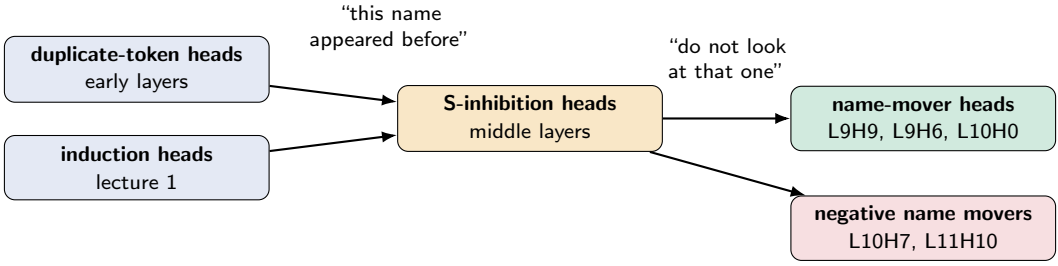
Suppress attention to it.

Copy whichever name is left.

Three steps. Notice the middle one: the model does not find the right answer directly. It finds the **wrong** one and rules it out.

That is not how a person would describe the task, and it is not what you would have guessed. It is what the interventions showed.

Who does what



The name movers are the only heads that write the answer. Everything upstream exists to tell them **which name not to copy**.

Two further classes - previous-token and backup name movers - are left off for space; the backups arrive in the next section.

What that cost

Model	GPT-2 small, 124 million parameters
Task	one sentence template
Result	26 heads, 7 classes
Effort	a team of researchers, months

That is the honest price of a complete circuit-level explanation, at the smallest scale anyone works at. Multiply by a frontier model and it does not survive contact.

Which is why the field moved on to the methods in the next lecture - not because circuits stopped being interesting, but because **finding them by hand does not scale**.

What goes wrong

The result that should make you distrust everything above

Predict first

We now know which three heads write the answer: **L9H9**, **L9H6**, **L10H0**. Their direct contributions add up to $+4.44$ of a $+3.58$ logit difference.

Delete all three. What is left?

near zero clearly negative barely changed

Attribution says the answer should collapse. Commit before the next slide.

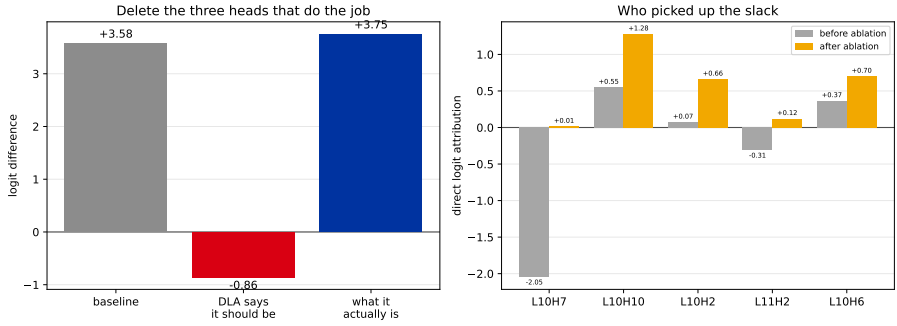
It gets better

baseline logit difference	$+3.580$
what attribution predicts after deleting the three	-0.858
what actually happens	$+\mathbf{3.754}$

We deleted the three heads that provably do the task, and the model got **slightly better at it**.

Every method in this lecture pointed at those three heads. Every method was right about what they contribute. And removing them changed almost nothing.

Where did +4.6 come from?



Left: prediction versus reality. Right: the direct contribution of five other heads, before and after the deletion.

Two mechanisms, not one

The +4.61 of compensation comes from two different places, and they are not the same kind of thing:

Source	Amount	What happened
L10H7 stops suppressing	+2.06 (45%)	the negative name mover: −2.05 → +0.02
backup heads step up	+1.85 (40%)	L10H10, L10H2, L10H6, L10H1 all increase

L10H7's job is to **suppress whatever the name movers boost** - so removing them leaves nothing to suppress and it goes quiet on its own. The network did not "repair" itself: **a brake came off at the same moment the engine was removed.** Only the second row is genuine redundancy.

The consequence

Ablation understates importance whenever a backup or a suppressor exists.
A component can be doing the work and measure as unimportant.

This is not a rare edge case. It happened on the first task we tried, in the most studied circuit in the field, with the three best-understood heads in the model.

Read every "we ablated X and nothing happened" with this in mind - including your own. Nothing happening is **not** evidence that X does nothing.

Named the **Hydra effect** by McGrath et al. (2023): cut off one head and another grows.

And it is not the only way to be fooled

- **Subspace patching illusions.** Patching inside a chosen subspace can produce a large, clean, entirely misleading effect - the direction you picked need not be the one the model uses. (Makelov et al.)
- **Metric dependence.** Logit difference, accuracy and loss can rank the same components differently. The circuit you find depends on the number you chose to watch.
- **Template dependence.** Our whole analysis used one sentence shape. Whether it survives a different phrasing is an empirical question, not an assumption.

Causal scrubbing (Redwood) is the field's most serious attempt at a rigorous standard for "is this explanation actually correct?" - it replaces everything the hypothesis says is irrelevant and checks the behaviour survives. Nobody finds it fully satisfying, which tells you where the field is.

One place where it all worked

There is a setting where a circuit was reverse-engineered **completely**: a small transformer trained to do modular addition. The mechanism turned out to be trigonometric - the model learned to represent numbers as angles and add them by rotation.

Fourier structure in the weights, an exact algorithm, ablations that behave. Everything this lecture wished for.

And the field has largely **retired** this kind of work - the author of that paper now lists toy-model interpretability as a direction to avoid, as too clean to say anything about real models. **Both are true:** it is the best way to *learn* what a circuit is, and not where answers about real models will come from.

Nanda et al. (2023); linked on the chapter page.

Recap

Method	Answers	Fails when
Ablation	is this component needed?	a backup or a suppressor exists
Activation patching	where does the information live?	you need components, not positions
Direct attribution	who writes to the output?	the component's work is indirect
Path patching	through which route?	the number of paths explodes
Attribution patching	all of the above, cheaply	the effect is large

We can now **test** a hypothesis about a component - and we have seen the tests disagree with each other on our own data. That is what having methods feels like.

Next

Everything today operated on **heads**, **layers** and **neurons** - the units the architecture happens to hand us.

We found one head doing the opposite of the task, four heads quietly waiting to take over, and a component that looks at the answer and contributes nothing.

What if heads and neurons are simply the **wrong units**?

Next: features. Superposition, sparse autoencoders, attribution graphs - and what interpretability is actually for.